
嵌入式系统实验指导书

[实验 1]

2025 年 9 月

目录

1. 实验一（LED 闪烁+按键实验）	1
1.1. LED 闪烁实验.....	1
1.1.1. 实验目的.....	1
1.1.2. 实验环境.....	1
1.1.3. 实验原理.....	1
1.1.4. 实验内容.....	5
1.1.5. 实验步骤.....	9
1.1.6. 实验现象.....	11
1.1.7. 思考题.....	11
1.2. 按键实验	11
1.2.1. 实验目的.....	12
1.2.2. 实验环境.....	12
1.2.3. 实验原理.....	12
1.2.4. 实验内容.....	15
1.2.5. 实验步骤.....	18
1.2.6. 实验现象.....	20
1.2.7. 思考题.....	20
2. 附录：源代码	21
2.1. LED 闪烁实验.....	21
2.2. 按键实验	25

1. 实验一（LED 闪烁+按键实验）

1.1. LED 闪烁实验

1.1.1. 实验目的

- 1) 了解和掌握如何配置和使用 STM32 的 GPIO 口
- 2) 掌握通过 STM32 GPIO 口控制 LED 灯点亮和熄灭的方法

1.1.2. 实验环境

- 1) 硬件：STM32 开发板、程序下载调试板、ARM JLINK 仿真器、USB 方口线、DC 5V 电源和 PC 机。

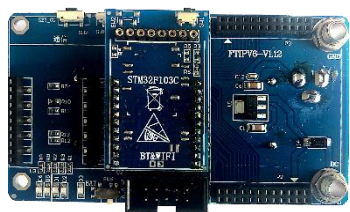


图 1-1 STM32 开发板

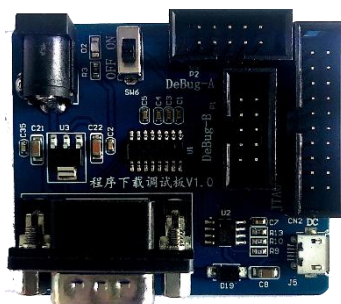


图 1-2 程序下载调试板



图 1-3 ARM JLINK 仿真器

- 2) 软件：Windows7 及以上系统、Keil5 MDK-ARM5.43a。

1.1.3. 实验原理

发光二极管的核心部分是由 p 型半导体和 n 型半导体组成的晶片，在 p 型半导体和 n 型半导体之间有一个过渡层，称为 p-n 结。在某些半导体材料的 PN 结中，注入的少数载流子与多数载流子复合时会把多余的能量以光的形式释放出来，从而把电能直接转换为光能。图 1-4 为 D1 灯和 D2 灯的原理图，当 IO 口为低电平时，LED 灯点亮。

D1 灯连接到了 GPIO 的 PA1 口；

D2 灯连接到了 GPIO 的 PA0 口。

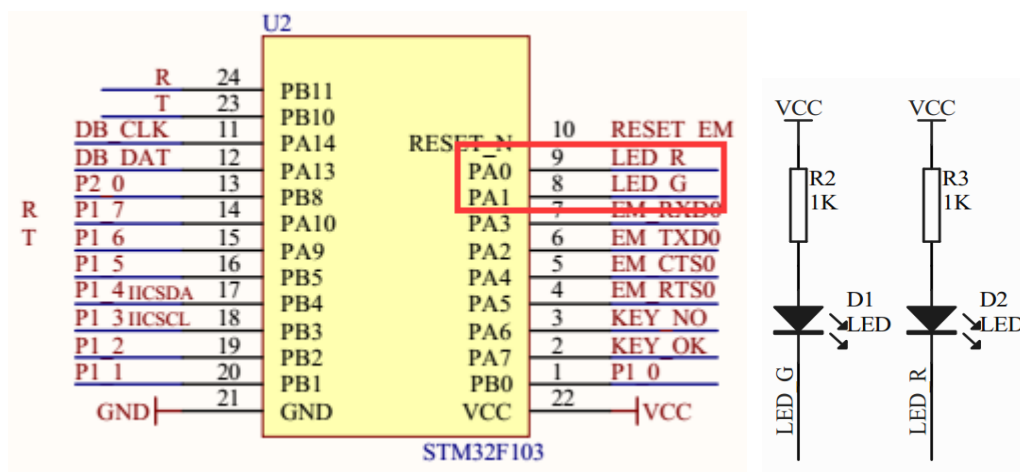


图 1-4 D1、D2 灯硬件原理图

首先先来介绍一下，在使用 STM32 进行编程时，主要使用库函数进行编程，在讲解固件库之前会先对重要寄存器进行一个讲解，这样是为了大家对寄存器有个初步的了解。大家学习固件库，并不需要记住每个寄存器的作用，而只是通过了解寄存器来对外设一些功能有个大致的了解，这样对以后的学习也很有帮助。

首先要提一下，在固件库中，GPIO 端口操作对应的库函数以及相关定义在文件 `stm32f10x_gpio.h` 和 `stm32f10x_gpio.c` 中。

STM32 的 IO 口相比 51 而言要复杂得多，所以使用起来也困难很多。STM32 的 IO 口可以由软件配置成如下 8 种模式：

- 1) 输入浮空
- 2) 输入上拉
- 3) 输入下拉
- 4) 模拟输入
- 5) 开漏输出
- 6) 推挽输出
- 7) 推挽式复用功能
- 8) 开漏复用功能

STM32 的每个 IO 端口都有 7 个寄存器来控制。他们分别是：配置模式的 2 个 32 位的端口配置寄存器 CRL 和 CRH；2 个 32 位的数据寄存器 IDR 和 ODR；1 个 32 位的置位/复位寄存器 BSRR；一个 16 位的复位寄存器 BRR；1 个 32 位的锁存寄存器 LCKR。这里我们仅介绍常用的几个寄存器，我们常用的 IO 端口寄存器只有 4 个：CRL、CRH、IDR、ODR。

STM32 的 IO 口位配置如表 1-1 所示：

表 1-1 STM32 的 IO 口位配置表

配置模式		CNF1	CNF0	MODE1	MODE0	PxODR寄存器		
通用输出	推挽式(Push-Pull)	0	0	01 10 11 见表 3. 1. 2		0 或 1		
	开漏(Open-Drain)		1			0 或 1		
复用功能输出	推挽式(Push-Pull)	1	0					不使用
	开漏(Open-Drain)		1					不使用
输入	模拟输入	0	0	00				不使用
	浮空输入		1					不使用
	下拉输入	1	0					0
	上拉输入							1

STM32 输出模式配置如表 1-2 所示：

表 1-2 STM32 输出模式配置表

MODE[1:0]	意义
00	保留
01	最大输出速度为10MHz
10	最大输出速度为2MHz
11	最大输出速度为50MHz

接下来我们看看端口低配置寄存器 CRL ， 如图 1-5 所示：

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
CNF7[1:0]		MODE7[1:0]		CNF6[1:0]		MODE6[1:0]		CNF5[1:0]		MODE5[1:0]		CNF4[1:0]		MODE4[1:0]	
RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
CNF3[1:0]		MODE3[1:0]		CNF2[1:0]		MODE2[1:0]		CNF1[1:0]		MODE1[1:0]		CNF0[1:0]		MODE0[1:0]	
RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW

位31:30	CNFy[1:0]: 端口x配置位(y = 0...7) 软件通过这些位配置相应的I/O端口，请参考表15端口位配置表。 在输入模式(MODE[1:0]=00): 00: 模拟输入模式 01: 浮空输入模式(复位后的状态) 10: 上拉/下拉输入模式 11: 保留 在输出模式(MODE[1:0]>00): 00: 通用推挽输出模式 01: 通用开漏输出模式 10: 复用功能推挽输出模式 11: 复用功能开漏输出模式
27:26	
23:22	
19:18	
15:14	
11:10	
7:6	
3:2	
位29:28	
25:24	
21:20	MODEy[1:0]: 端口x的模式位(y = 0...7) 软件通过这些位配置相应的I/O端口，请参考表15端口位配置表。 00: 输入模式(复位后的状态) 01: 输出模式，最大速度10MHz 10: 输出模式，最大速度2MHz 11: 输出模式，最大速度50MHz
17:16	
13:12	
9:8, 5:4	
1:0	

图 1-5 端口低配置寄存器 CRL 各位描述

IDR 是一个端口输入数据寄存器，只用了低 16 位。该寄存器为只读寄存器，并且只能以 16 位的形式读出。该寄存器描述如图 1-6 所示：



图 1-6 端口输入数据寄存器 IDR 各位描述

ODR 是一个端口输出数据寄存器，也只用了低 16 位。该寄存器为可读写，从该寄存器读出来的数据可以用于判断当前 IO 口的输出状态。而向该寄存器写数据，则可以控制某个 IO 口的输出电平。该寄存器的各位描述如图 1-7 所示：

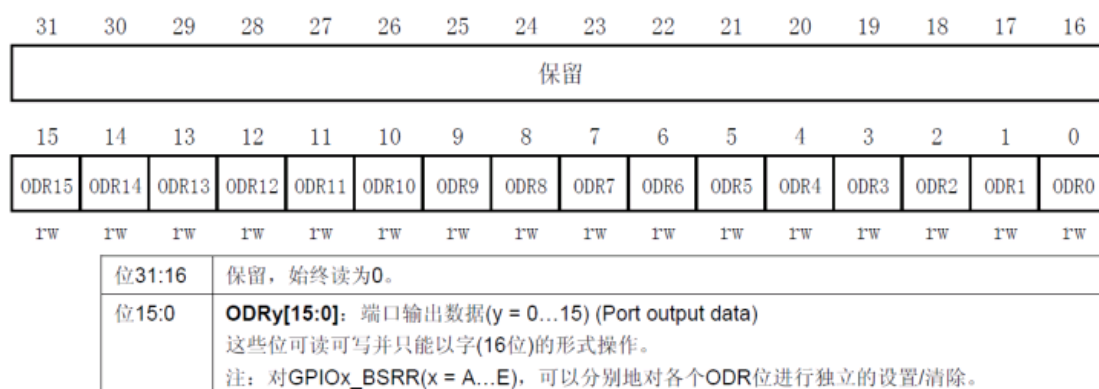


图 1-7 端口输出数据寄存器 ODR 各位描述

关于 IO 口更详细的介绍，请参考《STM32 中文参考手册》8.1 节。由于本次实验主要使用固件库开发，GPIO 相关的函数和定义分布在固件库文 stm32f10x_gpio.c 和头文件 stm32f10x_gpio.h 文件中。

在固件库开发中，操作寄存器 CRH 和 CRL 来配置 IO 口的模式和速度是通过 GPIO 初始化函数完成：

```
void GPIO_Init(GPIO_TypeDef* GPIOx, GPIO_InitTypeDef* GPIO_InitStruct);
```

这个函数有两个参数，第一个参数是用来指定 GPIO，取值范围为 GPIOA~GPIOG。第二个参数为初始化参数结构体指针，结构体类型为 GPIO_InitTypeDef。

```
typedef struct
{
    uint16_t GPIO_Pin;
    GPIO_Speed_TypeDef GPIO_Speed;
    GPIOMode_TypeDef GPIO_Mode;
```

```
}GPIO_InitTypeDef
```

通过初始化结构体初始化 GPIO 的常用格式是：

```
GPIO_InitTypeDef GPIO_InitStructure;  
GPIO_InitStructure.GPIO_Pin = GPIO_Pin_1; //D1-->PA.1 端口配  
GPIO_InitStructure.GPIO_Mode = GPIO_Mode_Out_PP; //推挽输出  
GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz; //速度 50MHz  
GPIO_Init(GPIOA, &GPIO_InitStructure); //根据设定参数配置 GPIO
```

这段 GPIO 初始化代码将 D1 灯设置为推挽输出，速度 50MHZ。

在 STM32 固件库中，对 LED 灯的控制是通过设置 BSRR 和 BRR 寄存器实现的。GPIO 端口输出是通过函数 GPIO_SetBits() 和函数 GPIO_ResetBits() 来完成的。

```
void GPIO_SetBits(GPIO_TypeDef* GPIOx, uint16_t GPIO_Pin);  
void GPIO_ResetBits(GPIO_TypeDef* GPIOx, uint16_t GPIO_Pin);
```

在多数情况下，我们都是采用这两个函数来设置 GPIO 端口的输出状态。比如我们要设置 GPIOA.0 输出 1，那么方法为：

```
GPIO_SetBits(GPIOA, GPIO_Pin_0);
```

反之如果要设置 GPIOA.0 输出 0，方法为：

```
GPIO_ResetBits (GPIOA, GPIO_Pin_0);
```

接下来我们来看一下，LED 闪烁实验的具体代码。

1.1.4. 实验内容

本次实验以 STM32 单片机为控制载体，通过控制 GPIO 实现 LED 的闪烁功能。

通过实验原理可知，本次实验的主要内容是配置 STM32 的 GPIO 口，GPIO 口配置成功后给它赋值，添加延时函数，实验 LED 闪烁功能。

1) 新建 led.c 文件，配置 LED 的引脚和端口（增加移植性）。

```
/*  
*****  
*****  
*/  
#include "led.h"  
#include "stm32f10x.h"  
#include "stm32f10x_gpio.h"  
#include "stm32f10x_rcc.h"  
  
#define D1 1  
#define D2 2  
  
//为了可移植性，将 LED 的引脚，端口，始终独立出来  
#define D1_GPIO_PIN GPIO_Pin_0  
#define D2_GPIO_PIN GPIO_Pin_1  
#define D1_GPIO_PORT GPIOA  
#define D2_GPIO_PORT GPIOA  
#define D1_GPIO_ACC RCC_APB2Periph_GPIOA  
#define D2_GPIO_ACC RCC_APB2Periph_GPIOA
```

2) LED 的初始化配置

```

void leds_init(void)
{
    GPIO_InitTypeDef GPIO_InitStructure;

    /* 初始化 GPIO 时钟 */
    RCC_APB2PeriphClockCmd(D1_GPIO_ACC | D2_GPIO_ACC, ENABLE);

    /* 设置 LED 的引脚为推挽输出 */
    GPIO_InitStructure.GPIO_Pin = D1_GPIO_PIN;
    GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;
    GPIO_InitStructure.GPIO_Mode = GPIO_Mode_Out_PP;
    GPIO_Init(D1_GPIO_PORT, &GPIO_InitStructure);

    GPIO_InitStructure.GPIO_Pin = D2_GPIO_PIN;
    GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;
    GPIO_InitStructure.GPIO_Mode = GPIO_Mode_Out_PP;
    GPIO_Init(D2_GPIO_PORT, &GPIO_InitStructure);

    //D1 和 D2 默认关闭
    D1_off();
    D2_off();
}

```

3) D1 与 D2 的点亮函数

```

void D1_on(void)
{
    GPIO_ResetBits(D1_GPIO_PORT, D1_GPIO_PIN);
}
void D2_on(void)
{
    GPIO_ResetBits(D2_GPIO_PORT, D2_GPIO_PIN);
}

```

4) D1 与 D2 的熄灭函数

```

void D1_off(void)
{
    GPIO_SetBits(D1_GPIO_PORT, D1_GPIO_PIN);
}
void D2_off(void)
{
    GPIO_SetBits(D2_GPIO_PORT, D2_GPIO_PIN);
}

```

5) D1 与 D2 的取反函数

```

void D1_toggle(void)
{
    GPIO_WriteBit(D1_GPIO_PORT, D1_GPIO_PIN, (BitAction)!GPIO_ReadOutputDataBit(D1_GPIO_PORT, D1_GPIO_PIN));
}
void D2_toggle(void)
{
    GPIO_WriteBit(D2_GPIO_PORT, D2_GPIO_PIN, (BitAction)!GPIO_ReadOutputDataBit(D2_GPIO_PORT, D2_GPIO_PIN));
}

```

6) 新建 led.h 文件


```

#ifndef LEDS_H
#define LEDS_H

void leds_init(void);
void D1_on(void);
void D2_on(void);
void D1_off(void);
void D2_off(void);
void D1_toggle(void);
void D2_toggle(void);
#endif

```

7) 新建 delay.c 文件

```

/*****      delay.c      *****/

#include "delay.h"
#include "misc.h"

static u8 fac_us = 0; //us 延时倍乘数
static u16 fac_ms = 0; //ms 延时倍乘数
//初始化延迟函数
//SYSTICK 的时钟固定为 HCLK 时钟的 1/8
//SYSCLK:系统时钟
void delay_init( u8 SYSCLK )
{
    SysTick_CLKSourceConfig( SysTick_CLKSource_HCLK_Div8 ); //选择外部时钟 HCLK/8
    fac_us = SYSCLK / 8;
    fac_ms = ( u16 )fac_us * 1000;
}
//延时 nms
//注意 nms 的范围
//SysTick->LOAD 为 24 位寄存器,所以,最大延时为:
//nms<=0xfffff*8*1000/SYSCLK
//SYSCLK 单位为 Hz,nms 单位为 ms
//对 72M 条件下,nms<=1864
void delay_ms( u16 nms )
{
    u32 temp;
    SysTick->LOAD = ( u32 )nms * fac_ms; //时间加载(SysTick->LOAD 为 24bit)
    SysTick->VAL = 0x00; //清空计数器
    SysTick->CTRL = 0x01; //开始倒数
    do
    {
        temp = SysTick->CTRL;
    }
    while( temp & 0x01 && !( temp & ( 1 << 16 ) ) ); //等待时间到达
    SysTick->CTRL = 0x00; //关闭计数器
    SysTick->VAL = 0x00; //清空计数器
}
//延时 nus
//nus 为要延时的 us 数.

void delay_us( u32 nus )
{
    u32 temp;

```

```

SysTick->LOAD = nus * fac_us; //时间加载
SysTick->VAL = 0x00; //清空计数器
SysTick->CTRL = 0x01; //开始倒数
do
{
    temp = SysTick->CTRL;
}
while( temp & 0x01 && !( temp & ( 1 << 16 ) ) ); //等待时间到达
SysTick->CTRL = 0x00; //关闭计数器
SysTick->VAL = 0x00; //清空计数器
}

```

8) 新建 delay.h 文件

```

/***** delay.h *****/

#ifndef __DELAY_H
#define __DELAY_H
#include "stm32f10x.h"

void delay_init(u8 SYSCLK);
void delay_ms(u16 nms);
void delay_us(u32 nus);

#endif

```

9) 打开 main.c 文件，配置主函数

```

#include <stdio.h>
#include "stm32f10x.h"
#include "led.h"
#include "delay.h"
int main(void)
{
    delay_init(72); //系统延时初始化，72 为芯片的主频，单位为 MHz
    leds_init(); //led 初始化
    D1_on(); //点亮 D1 灯

    while(1)
    {
        D1_toggle(); //D1 灯状态翻转
        D2_toggle(); //D2 灯状态翻转
        delay_ms(1000);
    }
    return 0;
}

```

以下是本节 LED 闪烁实验的流程图：

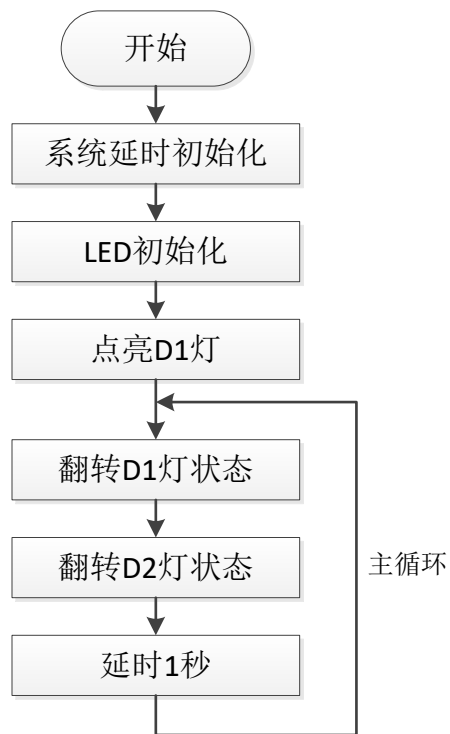


图 1-8 LED 闪烁实验流程图

1.1.5. 实验步骤

- 1) 使用 USB 方口线连接 ARM J-LINK 仿真器和 PC 机，用 20 针排线连接 ARM J-LINK 仿真器和下载调试板，10 针排线连接下载调试板和 STM32 开发板，如图 1-9 所示。



图 1-9 硬件连接图

- 2) 用 Keil5 开发环境打开实验例程，点击 Rebuild All 重新编译工程，如图 1-10 所示。

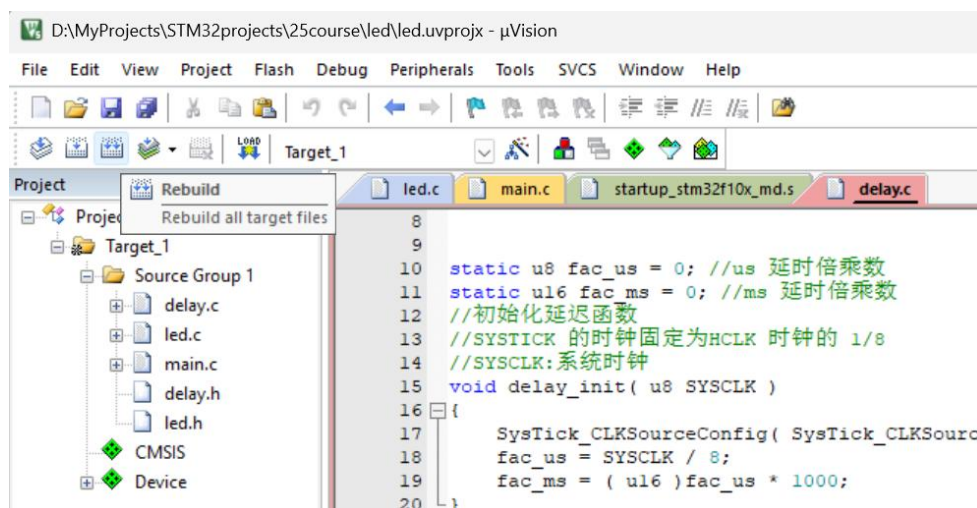


图 1-10 重新编译

- 3) 点击 Download 将程序下载到 STM32 开发板中，即可观察到开发板程序运行（如按照教程勾选“Reset and Run”）如图 1-11 所示。也可以将 STM32 开发板重新上电或者按下复位按钮让刚才下载的程序重新运行。

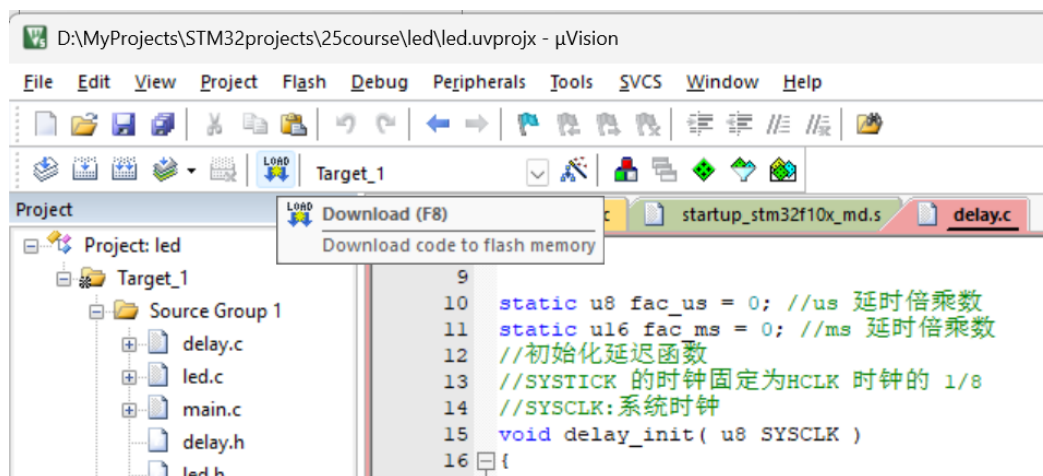


图 1-11 下载程序

- 4) （可选）下载完后可以点击 Debug 让程序进入调试模式，如图 1-12 所示；

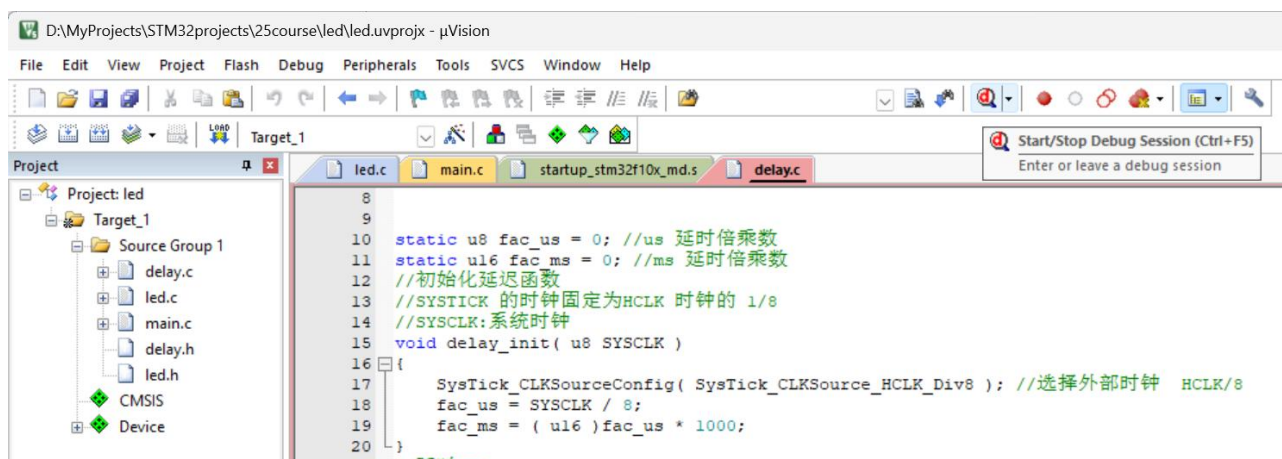


图 1-12 调试程序

5) 观察 STM32 开发板 LED 灯的现象。

1.1.6. 实验现象

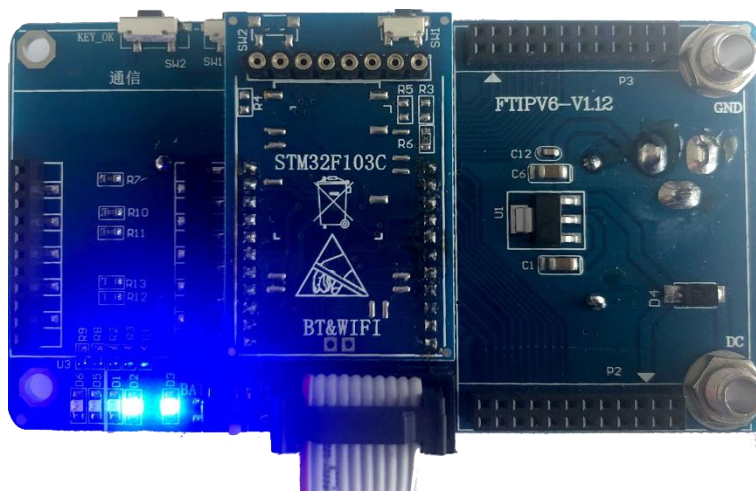


图 1-13 LED 闪烁实验现象

可以看到 STM32 开发板上的 D1 灯与 D2 灯交替闪烁，D1 灯和 D2 灯亮灭的时间间隔为 1s，如图 1-13 所示。

1.1.7. 思考题

若实现 D1、D2 交替闪烁 5 次以后，灭 5S，然后继续闪烁 5 次，灭 5S 依次循环。代码需要如何实现呢？

请思考并动手修改代码，实现以上功能。

1.2. 按键实验

1.2.1. 实验目的

- 1) 了解按键的工作原理及外部中断的基础知识
- 2) 掌握使用外部中断实现某些功能的方法

1.2.2. 实验环境

- 1) 硬件：STM32 开发板、程序下载调试板、ARM JLINK 仿真器、USB 方口线、DC 5V 电源和 PC 机。

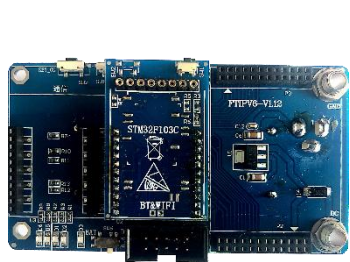


图 1-14 STM32 开发板

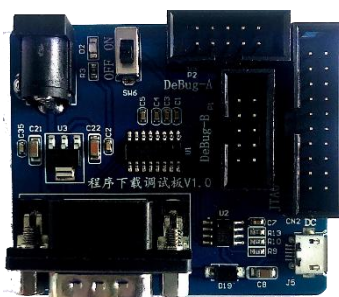


图 1-15 程序下载调试板



图 1-16 ARM JLINK 仿真器

- 2) 软件：Windows7 及以上系统、Keil5 MDK-ARM5.43a。

1.2.3. 实验原理

机械触点式按键开关，其主要功能是把机械上的通断转换为电气上的逻辑关系。它能提供标准的 TTL 逻辑电平，以便与通用数字系统的逻辑电平相容。对按键的响应取决于按键的工作方式，按键的工作方式应根据实际应用系统中的 CPU 的工作状况而定。选取的原则是既要保证 CPU 能及时响应按键操作，又不要过多占用 CPU 的工作时间。通常按键的工作方式有三种，编程扫描、定时扫描和中断扫描。我们采用中断扫描方式，可以提高 CPU 工作效率。图 1-17 为按键 KEY_OK(SW2)和 KEY_NO(SW1)的按键原理图。

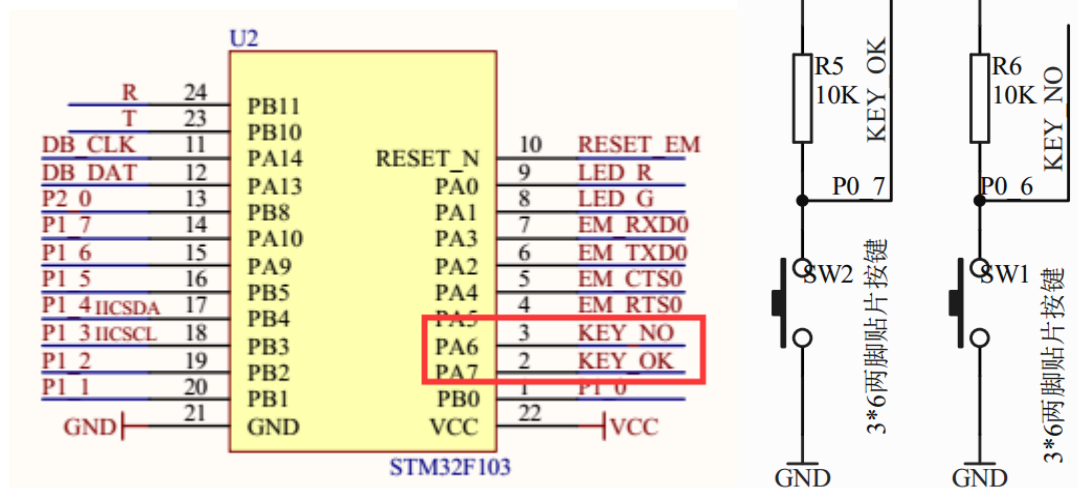


图 1-17 按键原理图

KEY_NO(SW1)连接到 STM32 的 PA6 口；

KEY_OK(SW2)连接到 STM32 的 PA7 口。

STM32F1 的 IO 口在 LED 闪烁实验中已经有了比较详细的介绍，这里我们不再多说。STM32F1 的 IO 口做输入使用的时候，是通过调用函数 `GPIO_ReadInputDataBit()` 来读取 IO 口的状态的。了解了这点，我们就先来看一下按键的外部中断实验需要使用哪些寄存器，这里我们将介绍 STM32 IO 口的中断功能，通过中断的功能，实现 LED 灯的控制，即：通过板载的 2 个按键 KEY_OK(SW2)和 KEY_NO(SW1)，来控制板载的 D1 和 D2 灯的亮灭。

本实验使用到的代码主要在固件库的 `stm32f10x_exti.h` 和 `stm32f10x_exti.c` 文件。

这里我们首先介绍一下 STM32 IO 口中断的一些基础概念。STM32 的每个 IO 口都可以作为外部中断的中断输入口，这点也是 STM32 的强大之处。STM32F103 的中断控制器支持 19 个外部中断/事件请求。每个中断设有状态位，每个中断/事件都有独立的触发和屏蔽设置。STM32F103 的 19 个外部中断为：

线 0~15：对应外部 IO 口的输入中断。

线 16：连接到 PVD 输出。

线 17：连接到 RTC 闹钟事件。

线 18：连接到 USB 唤醒事件。

从上面可以看出，STM32 供 IO 口使用的中断线只有 16 个，但是 STM32 的 IO 口却远远不止 16 个，那么 STM32 是怎么把 16 个中断线和 IO 口一一对应的呢？于是 STM32 就这样设计，GPIO 的管脚 `GPIOx.0~GPIOx.15` ($x=A,B,C,D,E, F,G$) 分别对应中断线 0~15。这样每个中断线对应了最多 7 个 IO 口，以线 0 为例：它对应了 `GPIOA.0`、`GPIOB.0`、`GPIOC.0`、`GPIOD.`

0、GPIOE.0、GPIOF.0、GPIOG.0。而中断线每次只能连接到 1 个 IO 口上，这样就需要通过配置来决定对应的中断线配置到哪个 GPIO 上了。

在库函数中，中断的初始化是通过函数 EXTI_Init()实现的。EXTI_Init()函数的定义是：

```
void EXTI_Init(EXTI_InitTypeDef* EXTI_InitStruct)
```

下面我们用一个使用范例来说明这个函数的使用：

```
EXTI_InitTypeDef EXTI_InitStructure;  
EXTI_InitStructure.EXTI_Line=EXTI_Line6;    //按键的 IO 口  
EXTI_InitStructure.EXTI_Mode = EXTI_Mode_Interrupt;  
EXTI_InitStructure.EXTI_Trigger = EXTI_Trigger_Falling;  
EXTI_InitStructure.EXTI_LineCmd = ENABLE;  
EXTI_Init(&EXTI_InitStructure); //根据 EXTI_InitStruct 中指定的  
//参数初始化外设 EXTI 寄存器
```

从上面的例子上我们可以看到，有 4 个参数我们进行了设置。第一个参数是中断线的标号，取值范围为 EXTI_Line0~EXTI_Line15，在上面已经讲过中断线的概念。也就是说，这个函数配置的是某个中断线上的中断参数。第二个参数是中断模式，可选值为中断 EXTI_Mode_Interrupt 和事件 EXTI_Mode_Event。第三个参数是触发方式，可以是下降沿触发 EXTI_Trigger_Falling，上升沿触发 EXTI_Trigger_Rising，或者任意电平（上升沿和下降沿）触发 EXTI_Trigger_Rising_Falling，最后一个参数就是使能中断线了。

我们设置好中断线和 GPIO 的映射关系，然后又设置好了中断的触发模式等初始化参数。既然是外部中断，涉及到中断我们当然还要设置 NVIC 中断优先级。这个在前面已经讲解过，这里我们就接着上面的范例，设置中断线 6 的中断优先级。

```
NVIC_InitTypeDef NVIC_InitStructure;  
NVIC_InitStructure.NVIC_IRQChannel = EXTI9_5_IRQn;  
NVIC_InitStructure.NVIC_IRQChannelPreemptionPriority = 0x03;  
NVIC_InitStructure.NVIC_IRQChannelSubPriority = 0x03;  
NVIC_InitStructure.NVIC_IRQChannelCmd = ENABLE;  
NVIC_Init(&NVIC_InitStructure);
```

设置中断优先级的步骤：

- 1) 定义个中断结构体，并配置 4 个参数，配置我们 IO 口所用的外部中断通道，设定抢占优先级和子优先级，并使能外部中断通道。
- 2) 设置所用到的中断的中断优先级别,对每个中断调用函数为 NVIC_Init();

寄存器配置详情请参考《STM32 中文参考手册》。

我们配置完中断优先级之后，接着要做的就是编写中断服务函数。中断服务函数的名字是在 MDK 中事先有定义的。这里需要说明一下，STM32 的 IO 口外部中断函数只有 6 个，分别为：

```
EXPORT EXTI0_IRQHandler
```



```
EXPORT EXTI1_IRQHandler
EXPORT EXTI2_IRQHandler
EXPORT EXTI3_IRQHandler
EXPORT EXTI4_IRQHandler
EXPORT EXTI9_5_IRQHandler
EXPORT EXTI15_10_IRQHandler
```

中断线 0-4 每个中断线对应一个中断函数，中断线 5-9 共用中断函数 EXTI9_5_IRQHandler，中断线 10-15 共用中断函数 EXTI15_10_IRQHandler。在编写中断服务函数的时候会经常使用到两个函数，第一个函数是判断某个中断线上的中断是否发生（标志位是否置位）：

```
void EXTI_GetFlagStatus (uint32_t EXTI_Line);
```

这个函数一般使用在中断服务函数的开头判断中断是否发生。另一个函数是清除某个中断线上的中断标志位：

```
void EXTI_ClearITPendingBit(uint32_t EXTI_Line);
```

这个函数一般应用在中断服务函数结束之前，清除中断标志位。常用的中断服务函数格式为：

```
void EXTI3_IRQHandler(void)
{
if(EXTI_GetFlagStatus (EXTI_Line6)!=RESET)//判断某个线上的中断是否发生
{
//中断逻辑...
EXTI_ClearITPendingBit(EXTI_Line6); //清除 LINE 上的中断标志位
}
}
```

下面我们再总结一下使用 IO 口外部中断的一般步骤：

- 1) 初始化 IO 口为输入；
- 2) 开启 GPIO 时钟；
- 3) 设置 IO 口与中断线的映射关系；
- 4) 初始化线上中断，设置触发条件等；
- 5) 配置中断分组（NVIC），并使能中断；
- 6) 编写中断服务函数。

通过以上几个步骤的设置，我们就可以正常使用外部中断了。接下来我们来看一下实验代码。

1.2.4. 实验内容

本次实验以 STM32 单片机为控制载体，通过按键产生外部中断实现对应的功能。

- 1) 在上一个实验的基础上，新建 key.c 文件，初始化按键 KEY_OK(SW2)和 KEY_NO(SW1)，并设置按键外部中断。

```

#include "key.h"
#include "led.h"
#include "stm32f10x_exti.h"
void exti_init(void)
{
    EXTI_InitTypeDef EXTI_InitStructure;
    GPIO_InitTypeDef GPIO_InitStructure;
    NVIC_InitTypeDef NVIC_InitStructure;

    /* 初始化 GPIOA 时钟 */
    RCC_APB2PeriphClockCmd(RCC_APB2Periph_GPIOA, ENABLE);

    /* 配置 PA.6 和 PA.7 脚为浮空输入 */
    GPIO_InitStructure.GPIO_Pin = GPIO_Pin_6 | GPIO_Pin_7;
    GPIO_InitStructure.GPIO_Mode = GPIO_Mode_IN_FLOATING;
    GPIO_Init(GPIOA, &GPIO_InitStructure);

    /* 将中断线 6 连接到 PA.6 pin */
    GPIO_EXTILineConfig(GPIO_PortSourceGPIOA, GPIO_PinSource6);
    /* 将中断线 7 连接到 PA.7 pin */
    GPIO_EXTILineConfig(GPIO_PortSourceGPIOA, GPIO_PinSource7);

    /* 初始化中断线 6、7 上的中断 */
    EXTI_InitStructure.EXTI_Line = EXTI_Line6 | EXTI_Line7;
    EXTI_InitStructure.EXTI_Mode = EXTI_Mode_Interrupt;
    EXTI_InitStructure.EXTI_Trigger = EXTI_Trigger_Falling;
    EXTI_InitStructure.EXTI_LineCmd = ENABLE;
    EXTI_Init(&EXTI_InitStructure);

    /* 设置中断线 9-5 的中断优先级 */
    NVIC_InitStructure.NVIC_IRQChannel = EXTI9_5_IRQn;
    NVIC_InitStructure.NVIC_IRQChannelPreemptionPriority = 0x03;
    NVIC_InitStructure.NVIC_IRQChannelSubPriority = 0x03;
    NVIC_InitStructure.NVIC_IRQChannelCmd = ENABLE;
    NVIC_Init(&NVIC_InitStructure);
}

```

- 2) 外部中断通道 5-9 中断处理程序，在程序里匹配对应的中断通道，并设置 LED 灯状态的翻转。

```

void EXTI9_5_IRQHandler(void)
{
    if(EXTI_GetFlagStatus(EXTI_Line7) != RESET)
    {
        D1_toggle();
        EXTI_ClearITPendingBit(EXTI_Line7);
    }
    if(EXTI_GetFlagStatus(EXTI_Line6) != RESET)
    {
        D2_toggle();
        EXTI_ClearITPendingBit(EXTI_Line6);
    }
}

```

- 3) 新建 key.h。

```
#ifndef __KEY_H
#define __KEY_H
#include "stm32f10x.h"

void exti_init(void);
void EXTI9_5_IRQHandler(void);

#endif
```

4) 打开 main.c 文件，配置主函数

```
/******
*****/
#include <stdio.h>
#include "stm32f10x.h"
#include "led.h"
#include "delay.h"
#include "key.h"
int main( void )
{
    delay_init( 72 ); //系统延时初始化，72 为芯片的主频，单位为 MHz
    NVIC_PriorityGroupConfig( NVIC_PriorityGroup_2 );
    leds_init(); //led 初始化
    exti_init();
    while( 1 )
    {

    }
    return 0;
}
```

接下来我们来看一下本节外部中断实验的流程图 1-18:

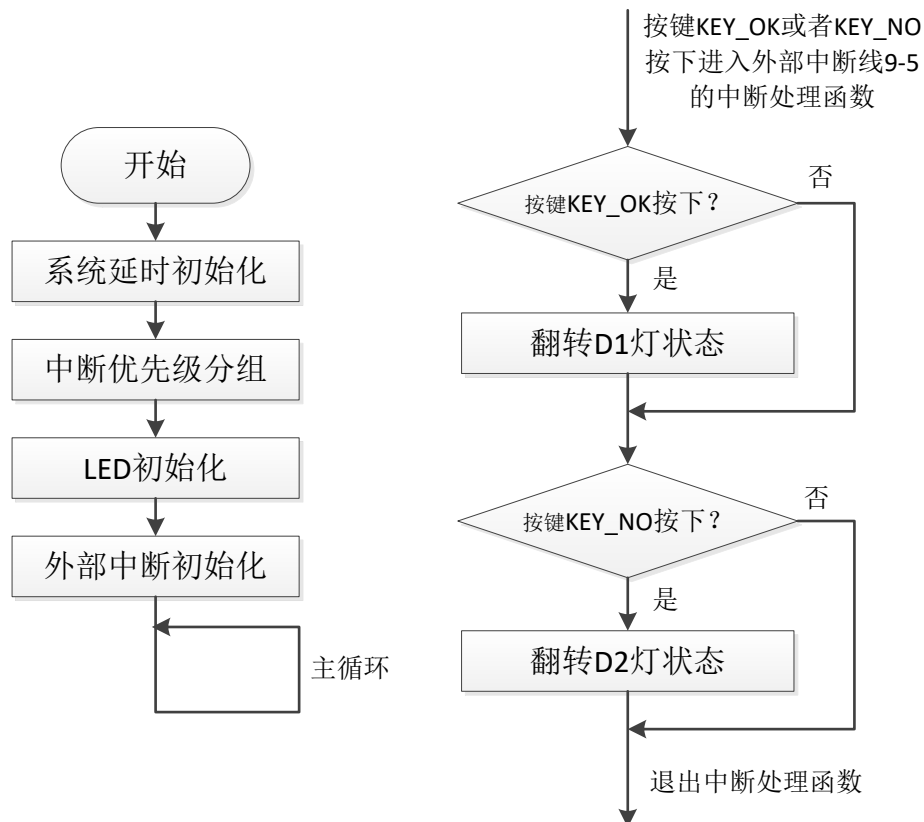


图 1-18 按键实验流程图

1.2.5. 实验步骤

- 1) 使用 USB 方口线连接 ARM J-LINK 仿真器和 PC 机，用 20 针排线连接 ARM J-LINK 仿真器和下载调试板，10 针排线连接下载调试板和 STM32 开发板，如图 1-19 所示。



图 1-19 硬件连接图

- 2) 用 Keil5 开发环境打开实验例程，点击 Rebuild All 重新编译工程，如图 1-20 所示。

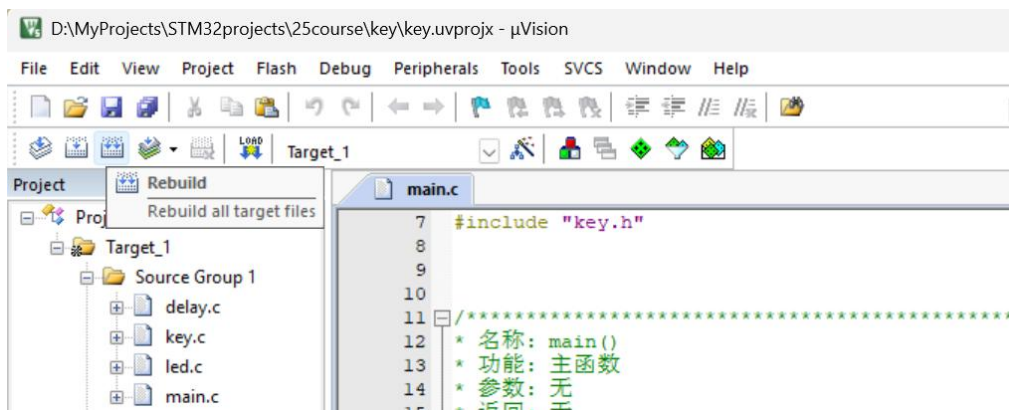


图 1-20 重新编译

- 3) 点击 **Download** 将程序下载到 STM32 开发板中，即可观察到开发板程序运行（如按照教程勾选“Reset and Run”）如图 1-21 所示。也可以将 STM32 开发板重新上电或者按下复位按钮让刚才下载的程序重新运行。

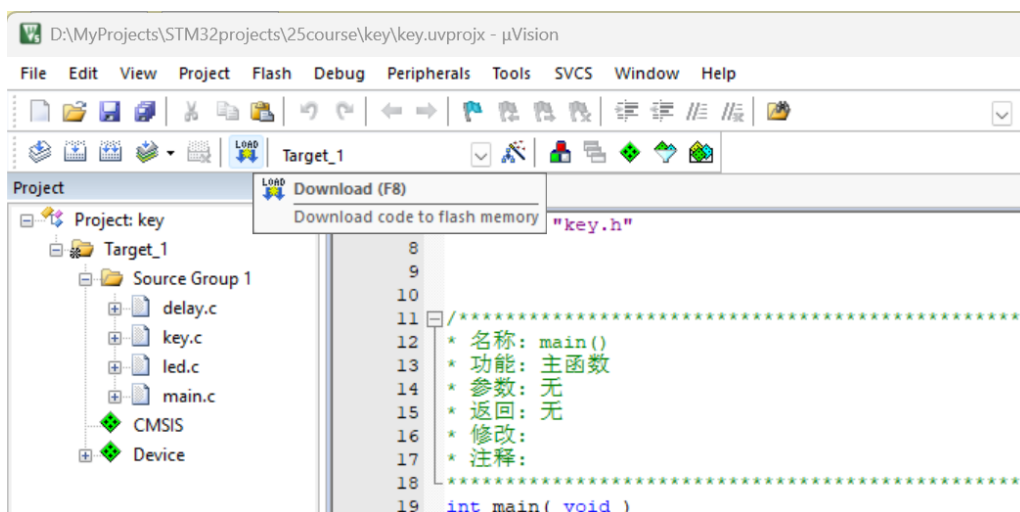


图 1-21 下载程序

- 4) （可选）下载完后可以点击 **Debug** 让程序进入调试模式，如图 1-22 所示；

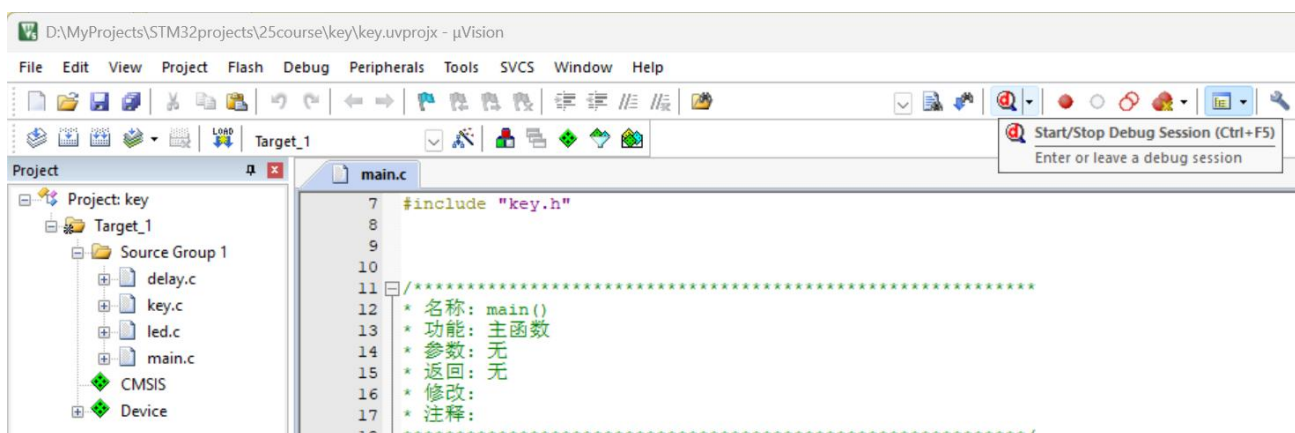


图 1-22 调试程序

- 5) 分别按下 KEY_OK（SW2）和 KEY_NO（SW1）键观察灯 D1 和 D2 的现象。

1.2.6. 实验现象

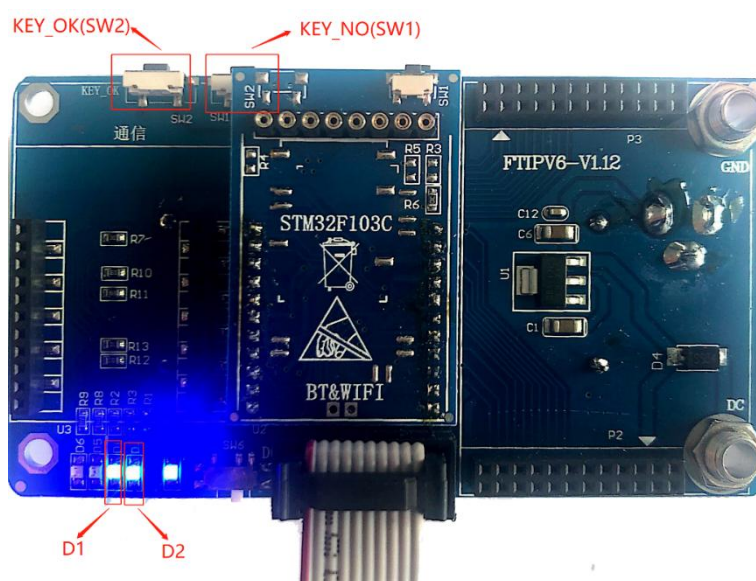


图 1-23 按键实验现象

可以看到当按下 KEY_OK (SW2) 时可以控制 D1 灯的亮灭，当按下 KEY_NO (SW1) 时可以控制 D2 灯的亮灭，如图 1-23 所示。

1.2.7. 思考题

若实现按下 KEY_OK (SW2) 时 D1、D2 全亮，当按下 KEY_NO (SW1) 时 D1、D2 全灭。代码需要如何实现呢？

请思考并动手修改代码，实现以上功能。

2. 附录：源代码

2.1. LED 闪烁实验

1) led.h

```

/*****
led.h
*****/

#ifndef LEDS_H
#define LEDS_H

void leds_init(void);
void D1_on(void);
void D2_on(void);
void D1_off(void);
void D2_off(void);
void D1_toggle(void);
void D2_toggle(void);
#endif

```

2) led.c

```

/*****
#include "led.h"
#include "stm32f10x.h"
#include "stm32f10x_gpio.h"
#include "stm32f10x_rcc.h"

#define D1 1
#define D2 2

//为了可移植性，将 LED 的引脚，端口，始终独立出来
#define D1_GPIO_PIN GPIO_Pin_0
#define D2_GPIO_PIN GPIO_Pin_1
#define D1_GPIO_PORT GPIOA
#define D2_GPIO_PORT GPIOA
#define D1_GPIO_ACC RCC_APB2Periph_GPIOA
#define D2_GPIO_ACC RCC_APB2Periph_GPIOA

/*****
* 名称: leds_init()
* 功能: 初始化 led 的 GPIO 时钟，并初始化为输出模式
* 参数: 无
* 返回: 无
* 修改:
* 注释: 1) 在使用 stm32 的 led 前，必须先调用这个初始化函数
*****/

void leds_init( void )
{
    GPIO_InitTypeDef GPIO_InitStructure;

```

```

/* 初始化 GPIO 时钟 */
RCC_APB2PeriphClockCmd( D1_GPIO_ACC | D2_GPIO_ACC, ENABLE );

/* 设置 LED 的引脚为推挽输出 */
GPIO_InitStructure.GPIO_Pin = D1_GPIO_PIN;
GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;
GPIO_InitStructure.GPIO_Mode = GPIO_Mode_Out_PP;
GPIO_Init( D1_GPIO_PORT, &GPIO_InitStructure );

GPIO_InitStructure.GPIO_Pin = D2_GPIO_PIN;
GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;
GPIO_InitStructure.GPIO_Mode = GPIO_Mode_Out_PP;
GPIO_Init( D2_GPIO_PORT, &GPIO_InitStructure );

//D4 和 D5 默认关闭
D1_off();
D2_off();
}

/*****
* 名称: D4_on()
* 功能: 点亮 D4
* 参数: 无
* 返回: 无
* 修改:
* 注释:
*****/
void D1_off( void )
{
    GPIO_SetBits( D1_GPIO_PORT, D1_GPIO_PIN );
}

/*****
* 名称: D5_on()
* 功能: 点亮 D5
* 参数: 无
* 返回: 无
* 修改:
* 注释:
*****/
void D2_off( void )
{
    GPIO_SetBits( D2_GPIO_PORT, D2_GPIO_PIN );
}

/*****
* 名称: D4_off()
* 功能: 熄灭 D4
* 参数: 无
* 返回: 无
* 修改:
* 注释:
*****/
void D1_on( void )
{

```



```

    GPIO_ResetBits( D1_GPIO_PORT, D1_GPIO_PIN );
}

/*****
* 名称: D5_off()
* 功能: 熄灭 D5
* 参数: 无
* 返回: 无
* 修改:
* 注释:
*****/
void D2_on( void )
{
    GPIO_ResetBits( D2_GPIO_PORT, D2_GPIO_PIN );
}

/*****
* 名称: D4_toggle()
* 功能: 反转 D4
* 参数: 无
* 返回: 无
* 修改:
* 注释:
*****/
void D1_toggle( void )
{
    GPIO_WriteBit( D1_GPIO_PORT, D1_GPIO_PIN, ( BitAction )!GPIO_ReadOutputDataBit( D1_GPIO_PORT,
D1_GPIO_PIN ) );
}

/*****
* 名称: D5_toggle()
* 功能: 反转 D5
* 参数: 无
* 返回: 无
* 修改:
* 注释:
*****/
void D2_toggle( void )
{
    GPIO_WriteBit( D2_GPIO_PORT, D2_GPIO_PIN, ( BitAction )!GPIO_ReadOutputDataBit( D2_GPIO_PORT,
D2_GPIO_PIN ) );
}

```

3) delay.h

```

/*****      delay.h      *****/

#ifndef __DELAY_H
#define __DELAY_H
#include "stm32f10x.h"

void delay_init(u8 SYSCLK);
void delay_ms(u16 nms);
void delay_us(u32 nus);
#endif

```

4) delay.c

```
/****** delay.c *****/

#include "delay.h"
#include "misc.h"

static u8 fac_us = 0; //us 延时倍乘数
static u16 fac_ms = 0; //ms 延时倍乘数
//初始化延迟函数
//SYSTICK 的时钟固定为 HCLK 时钟的 1/8
//SYSCLK:系统时钟
void delay_init( u8 SYSCLK )
{
    SysTick_CLKSourceConfig( SysTick_CLKSource_HCLK_Div8 ); //选择外部时钟 HCLK/8
    fac_us = SYSCLK / 8;
    fac_ms = ( u16 )fac_us * 1000;
}
//延时 nms
//注意 nms 的范围
//SysTick->LOAD 为 24 位寄存器,所以,最大延时为:
//nms<=0xfffff*8*1000/SYSCLK
//SYSCLK 单位为 Hz,nms 单位为 ms
//对 72M 条件下,nms<=1864
void delay_ms( u16 nms )
{
    u32 temp;
    SysTick->LOAD = ( u32 )nms * fac_ms; //时间加载(SysTick->LOAD 为 24bit)
    SysTick->VAL = 0x00; //清空计数器
    SysTick->CTRL = 0x01; //开始倒数
    do
    {
        temp = SysTick->CTRL;
    }
    while( temp & 0x01 && !( temp & ( 1 << 16 ) ) ); //等待时间到达
    SysTick->CTRL = 0x00; //关闭计数器
    SysTick->VAL = 0x00; //清空计数器
}
//延时 nus
//nus 为要延时的 us 数.
void delay_us( u32 nus )
{
    u32 temp;
    SysTick->LOAD = nus * fac_us; //时间加载
    SysTick->VAL = 0x00; //清空计数器
    SysTick->CTRL = 0x01; //开始倒数
    do
    {
        temp = SysTick->CTRL;
    }
    while( temp & 0x01 && !( temp & ( 1 << 16 ) ) ); //等待时间到达
    SysTick->CTRL = 0x00; //关闭计数器
    SysTick->VAL = 0x00; //清空计数器
}
```

```
}
```

5) main.c

```
/******  
*****/  
#include <stdio.h>  
#include "stm32f10x.h"  
#include "led.h"  
#include "delay.h"  
  
/******  
* 名称: main()  
* 功能: 主函数  
* 参数: 无  
* 返回: 无  
* 修改:  
* 注释:  
*****/  
int main( void )  
{  
  
    delay_init( 72 ); //系统延时初始化, 72 为芯片的主频, 单位为 MHz  
    leds_init(); //led 初始化  
    D1_on();  
  
    while( 1 )  
    {  
        for( int i = 0; i < 5; i++ )  
        {  
            D1_toggle(); //D1 灯状态翻转  
            D2_toggle(); //D2 灯状态翻转  
            delay_ms( 500 );  
        }  
        D1_off();  
        D2_off();  
        delay_ms( 5000 );  
    }  
}
```

2.2. 按键实验

1) key.c

```
#include "key.h"  
#include "led.h"  
#include "stm32f10x_exti.h"  
  
void exti_init( void )  
{
```

```

EXTI_InitTypeDef EXTI_InitStructure;
GPIO_InitTypeDef GPIO_InitStructure;
NVIC_InitTypeDef NVIC_InitStructure;

/* 初始化 GPIOA 时钟 */
RCC_APB2PeriphClockCmd( RCC_APB2Periph_GPIOA, ENABLE );

/* 配置 PA.6 和 PA.7 脚为浮空输入 */
GPIO_InitStructure.GPIO_Pin = GPIO_Pin_6 | GPIO_Pin_7;
GPIO_InitStructure.GPIO_Mode = GPIO_Mode_IN_FLOATING;
GPIO_Init( GPIOA, &GPIO_InitStructure );

/* 将中断线 6 连接到 PA.6 pin */
GPIO_EXTILineConfig( GPIO_PortSourceGPIOA, GPIO_PinSource6 );
/* 将中断线 7 连接到 PA.7 pin */
GPIO_EXTILineConfig( GPIO_PortSourceGPIOA, GPIO_PinSource7 );

/* 初始化中断线 6、7 上的中断 */
EXTI_InitStructure.EXTI_Line = EXTI_Line6 | EXTI_Line7;
EXTI_InitStructure.EXTI_Mode = EXTI_Mode_Interrupt;
EXTI_InitStructure.EXTI_Trigger = EXTI_Trigger_Falling;
EXTI_InitStructure.EXTI_LineCmd = ENABLE;
EXTI_Init( &EXTI_InitStructure );

/* 设置中断线 9-5 的中断优先级 */
NVIC_InitStructure.NVIC_IRQChannel = EXTI9_5_IRQn;
NVIC_InitStructure.NVIC_IRQChannelPreemptionPriority = 0x03;
NVIC_InitStructure.NVIC_IRQChannelSubPriority = 0x03;
NVIC_InitStructure.NVIC_IRQChannelCmd = ENABLE;
NVIC_Init( &NVIC_InitStructure );
}

void EXTI9_5_IRQHandler( void )
{
    if( EXTI_GetFlagStatus( EXTI_Line7 ) != RESET )
    {
        D1_on();
        D2_on();
        EXTI_ClearITPendingBit( EXTI_Line7 );
    }
    if( EXTI_GetFlagStatus( EXTI_Line6 ) != RESET )
    {
        D1_off();
        D2_off();
        EXTI_ClearITPendingBit( EXTI_Line6 );
    }
}

```

2) key.h

```

#ifndef __KEY_H
#define __KEY_H
#include "stm32f10x.h"
void exti_init(void);
void EXTI9_5_IRQHandler(void);

```

```
#endif
```

```
3) led.c (同上一节)
```

```
4) led.h (同上一节)
```

```
5) delay.c (同上一节)
```

```
6) delay.h (同上一节)
```

```
7) main.c
```

```
/*
*****
*****
*/
#include <stdio.h>
#include "stm32f10x.h"
#include "led.h"
#include "delay.h"
#include "key.h"

/*
*****
* 名称: main()
* 功能: 主函数
* 参数: 无
* 返回: 无
* 修改:
* 注释:
*****
*/
int main( void )
{
    delay_init( 72 ); //系统延时初始化, 72 为芯片的主频, 单位为 MHz
    NVIC_PriorityGroupConfig( NVIC_PriorityGroup_2 );
    leds_init(); //led 初始化
    exti_init();
    while( 1 )
    {

    }

    return 0;
}
```